instituto superior de engenharia de lisboa

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Primeiro Trabalho Prático

51777 : Bernardo Charneca (a51777@alunos.isel.pt)

51744 : Daniel Barreiro (a51744@alunos.isel.pt)

51447 : Ricardo Pinto (a51447@alunos.isel.pt)

Relatório para a Unidade Curricular de Lógica e Computação
da Licenciatura em Engenharia Informática e de Computadores

Professor : Dr. Nuno Leite

29 – novembro – 2025

Resumo

Este trabalho tem como objetivos: compreender a dedução automática no cálculo de predicados; compreender o mecanismo de dedução do PROLOG; compreender o efeito do operador cut, !, em programas PROLOG; compreender e dominar a sintaxe de programas PROLOG; compreender e dominar os construtos básicos de programação em PROLOG, tais como átomos, números, termos compostos, listas, operadores e aritmética; compreender e controlar o retrocesso (backtracking) e saber utilizar predicados embutidos típicos. Para atingir os objetivos propostos, foi realizada a implementação do jogo CONNECT 4 em PROLOG.

Resume

This work has the following objectives: understand automatic deduction in predicate calculus; understand the PROLOG deduction mechanism; understand the effect of the cut operator, !, in PROLOG programs; understand and master the syntax of PROLOG programs; understand and master the basic programming constructs of PROLOG, such as atoms, numbers, compound terms, lists, operators, and arithmetic; understand and control backtracking and know how to use typical built-in predicates. To achieve the proposed objectives the implementation of the CONNECT 4 game in PROLOG was carried out.

Índice

1 Connect 4 **1**

1.1 Explicação das funções do ficheiro connect4.pl 1

1.1.1 Conclusão 6

1

Connect 4

1.1 Explicação das funções do ficheiro connect4.pl

```
createBoard(Board) :- length(Row, 7),  maplist(=('-'), Row),
                      length(Board, 6), maplist(=(Row), Board).
```

A função ***createBoard*** serve para a criação do tabuleiro usando 6 listas de 7 elementos iniciados com '-'.

```
printIndices :- write('  0   1   2   3   4   5   6'), nl.
printTopBottomBar :- write('+---+---+---+---+---+---+---+'), nl.
printRow([]) :- write('|').
printRow([Head|Tail]) :- write('| '),
                         write(Head), write(' '), printRow(Tail).
```

As funções ***printIndices***, ***printTopBottomBar***, ***printRow*** auxiliam no desenho do tabuleiro para que ele não seja mostrado em forma de listas.

```
printBoard([]) :- printTopBottomBar.
printBoard([Row|Tail]) :- printTopBottomBar, printRow(Row), nl,
                          printBoard(Tail).
showBoard(Board) :- printIndices,
                   printBoard(Board), !.
```

As funções ***printBoard*** e ***showBoard*** servem para combinar as funções *printIndices*, *printTopBottomBar*, *printRow* de modo a serem chamadas juntamente para facilitar o seu uso.

```
isEmpty(Board, RowIdx, ColIdx) :-
  nth0(RowIdx, Board, Row),
  nth0(ColIdx, Row, Value),
  Value = '-'.
```

A função ***isEmpty*** serve para verificar se o lugar do tabuleiro está ocupado ou livre. É usado **nth0** para obter a row e depois o espaço do tabuleiro pretendido para jogar e depois verifica se está livre ou não.

```
board_full(Board) :-
  nth0(0, Board, TopRow),
  \+ member('-', TopRow).
```

A função ***board_full*** serve para verificar se o jogo empatou quando o tabuleiro fica cheio e sem vencedor. É obtido a row de cima do tabuleiro para a lógica do '-' não pertencer na lista, significando que o tabuleiro está cheio.

```
setPiece(Board, RowIdx, ColIdx, Value, NewBoard) :-
  nth0(RowIdx, Board, OldRow, OtherRows),
  nth0(ColIdx, OldRow, _OldCell, RowRest),
  nth0(ColIdx, NewRow, Value, RowRest),
  nth0(RowIdx, NewBoard, NewRow, OtherRows).
```

A função ***setPiece*** serve para modificar o tabuleiro ao meter a peça. Ela apanha a row de onde é planeado jogar, guardando o resto das rows. Guarda o resto dos espaços naquela row, cria uma nova row com a jogada e os espaços guardados anteriormente e, finalmente, ela cria um novo tabuleiro com a nova row e as rows inicialmente guardadas.

```
playPiece(Board, RowIdx, ColIdx, Value, NewBoard, RowIdx) :-
  isEmpty(Board, RowIdx, ColIdx),
  setPiece(Board, RowIdx, ColIdx, Value, NewBoard), !.

playPiece(Board, RowIdx, ColIdx, Value, NewBoard, RowPlaced) :-
  NewRowIdx is RowIdx - 1,
  playPiece(Board, NewRowIdx, ColIdx, Value, NewBoard, RowPlaced).
```

A função ***playPiece*** serve para decidir qual row é possível jogar a peça, em que este sobe a row caso o lugar esteja ocupado . Ela verifica se o lugar está vazio. Se estiver é jogado, senão ele vai para a sua segunda função em que ele vai subir a row index.

```
play(Board, Col, Value, NewBoard, RowPlaced) :-
  playPiece(Board, 5, Col, Value, NewBoard, RowPlaced).
```

A função ***play*** serve para a lógica das peças começar na última row do tabuleiro e, assim, simplificando as funções usadas para facilitar o uso dela.

```
get_cell(Board, Row, Col, Value) :-
  Row >= 0, Row < 6,
  Col >= 0, Col < 7,
  nth0(Row, Board, R),
  nth0(Col, R, Value).
```

A função ***get_cell*** é utilizada para obter o valor presente numa posição específica do tabuleiro, servindo de apoio às funções que verificam condições de vitória. Ela, primeiro, confirma que a posição indicada encontra-se dentro dos limites do tabuleiro. De seguida, obtém a row correspondente e, por fim, o valor da célula nessa posição, permitindo verificar se corresponde à peça procurada.

```
count_direction(Board, Row, Col, Player, DRow, DCol, Count) :-
  NextRow is Row + DRow,
  NextCol is Col + DCol,
  get_cell(Board, NextRow, NextCol, Player), !,
  count_direction(Board, NextRow, NextCol,
    Player, DRow, DCol, TailCount),
  Count is TailCount + 1.

count_direction(_Board, _Row, _Col, _Player, _DRow, _DCol, 0).
```

A função ***count_direction*** serve para contar quantas peças numa row de qualquer direção são do jogador para uso nas funções de verificação de vitória. Ela através do Count, NextRow e NextCol fica a ir numa row reta de uma certa direção a contar quantas peças seguidas são do jogador com o uso de *get_cell*, caso *get_cell* falhe ou o lugar escolhido estiver fora do tabuleiro ou a peça daquele lugar não é do jogador, ele passa para a segunda função que apenas retorna o Count a 0.

```
check_horizontal(Board, Row, Col, Player) :-
  count_direction(Board, Row, Col, Player, 0, -1, Left),
  count_direction(Board, Row, Col, Player, 0, +1, Right),
  Total is Left + 1 + Right,
  Total >= 4.

check_vertical(Board, Row, Col, Player) :-
  count_direction(Board, Row, Col, Player, +1, 0, Down),
  Total is Down + 1,
  Total >= 4.

check_diag1(Board, Row, Col, Player) :-
  count_direction(Board, Row, Col, Player, -1, -1, UpLeft),
  count_direction(Board, Row, Col, Player, +1, +1, DownRight),
  Total is UpLeft + 1 + DownRight,
  Total >= 4.

check_diag2(Board, Row, Col, Player) :-
  count_direction(Board, Row, Col, Player, -1, +1, UpRight),
  count_direction(Board, Row, Col, Player, +1, -1, DownLeft),
  Total is UpRight + 1 + DownLeft,
  Total >= 4.
```

Todas as funções de verificação contam peças iguais consecutivas nas duas direções e verificam se o total é >= 4. As funções ***check_horizontal***, ***check_vertical***, ***check_diag1*** e ***check_diag2*** servem

para verificar se existe vitória na horizontal, na vertical ou diagonais, respetivamente, no tabuleiro. Elas chamam a função count_direction para verificar os dois lados, juntando o Count delas para verificar se a soma delas é acima de 3, ou seja vitória.

```
check_win(Board, Row, Col, Player) :-
  check_horizontal(Board, Row, Col, Player);
  check_vertical(Board, Row, Col, Player);
  check_diag1(Board, Row, Col, Player);
  check_diag2(Board, Row, Col, Player).
```

A função ***check_win*** usa as funções: *check_horizontal*, *check_vertical*, *check_diag1* e *check_diag2* de modo usá-las numa só função.

```
validPlay(Col) :- Col >= 0, Col =< 6.
```

A função ***validPlay*** serve para verificar se a column digitada pelo jogador está dentro da column.

```
turnPlayer1(Board) :-
  write('Player 1 Turn (X): Select a Column (0 to 6)'), nl,
  read(Col1),
  ( validPlay(Col1) ->
      ( isEmpty(Board, 0, Col1) ->
          play(Board, Col1, 'X', Board1, Row1),
          showBoard(Board1),
          ( check_win(Board1, Row1, Col1, 'X') ->
              writeln('Player 1 (X) wins'),
              playAgain
          ;   board_full(Board1) ->
              writeln('Draw! Board full.'),
              playAgain
          ;   turnPlayer2(Board1)
          )
      ;   writeln('Column is full!'), nl,
          showBoard(Board),
          turnPlayer1(Board)
      )
  ;   writeln('Invalid column (0 to 6 only)'),
      showBoard(Board),
      turnPlayer1(Board)
  ).

turnPlayer2(Board) :-
  write('Player 2 Turn (O): Select a Column (0 to 6)'), nl,
  read(Col2),
  ( validPlay(Col2) ->
```

```
        ( isEmpty(Board, 0, Col2) ->
            play(Board, Col2, 'O', Board2, Row2),
            showBoard(Board2),
            ( check_win(Board2, Row2, Col2, 'O') ->
                writeln('Player 2 (O) wins'),
                playAgain
            ;   board_full(Board2) ->
                writeln('Draw! Board full.'),
                playAgain
            ;   turnPlayer1(Board2)
            )
        ;   writeln('Column is full!'), nl,
            showBoard(Board),
            turnPlayer2(Board)
        )
    ;   writeln('Invalid column (0 to 6 only)'),
        showBoard(Board),
        turnPlayer2(Board)
    ).
```

A função ***turnPlayer1*** serve: para executar as jogadas do jogador1, verificando se a jogada dele é valida; jogar; verificar vitória ou empate e passar o turno para o outro jogador. Ela começa por receber a column que é pretendida jogar para verificar se a column é valida e não está cheia. Caso seja um desses casos vai escrever o tipo de erro, mostrar novamente o tabuleiro e reiniciar a função para que o jogador possa executar uma jogada válida. Caso a jogada seja válida é jogado com a função play a peça do jogador e mostrado o novo tabuleiro. Depois é verificado se houve vitória ou empate. Caso tenha ocorrido é mandado para a função *playAgain* para tratar do próximo jogo, caso não ele passa a jogada para o outro jogador. ***turnPlayer2*** funciona de igual modo, só que para o jogador2.

```
playAgain :-
  write('Rematch? (y/n)'), nl,
  read(Response),
  ( Response = 'y' ->
      connect4
  ; Response = 'n' ->
      writeln('Game Over'), !
  ;   writeln('Invalid response (y/n only)'),
      playAgain
  ).
```

A função ***playAgain*** serve para quando o jogo anterior é acabado, deixa os jogadores decidirem se querem jogar novamente sem a necessidade de chamar a função *connect4* novamente. Ela espera por uma resposta entre **y** ou **n**, caso seja **y** ela chama a função *connect4* pelo jogador, caso seja **n** ela para a função e caso seja nenhum deles, ele escreve que é apenas aceite y ou n e volta a perguntar ao chamar a função *playAgain*.

```
connect4 :- createBoard(Board),
  showBoard(Board),
  turnPlayer1(Board).
```

A função ***connect4*** serve para simplificar as funções necessárias para jogar o jogo. Ela chama a função *createBoard* para criar o tabuleiro e chama a função *turnPlayer1* com o tabuleiro criado para iniciar o jogo.

Nota:

O código completo encontra-se no ficheiro anexo *connect4.pl*.

1.1.1 Conclusão

O jogo Connect 4 foi implementado em PROLOG com sucesso, permitindo que dois jogadores joguem alternadamente, com verificação de vitórias e empates, bem como a opção de jogar novamente após o término de uma partida.